

Question 1

1. Algorithm

1. Start

2. Declare integer variables `rows`, `cols`, `i`, `j`, and `k`.

3. Input Validation for Rows:

- Prompt the user to enter the number of rows.
- Check if `rows <= 0`. If true, display an error message and repeat the prompt until a positive integer is entered.

4. Input Validation for Columns:

- Prompt the user to enter the number of columns.
- Check if `cols <= 0`. If true, display an error message and repeat the prompt until a positive integer is entered.

5. **Array Declaration:** Declare a 2D variable-length array `arr[rows][cols]`.

6. Column-Major Order Input Reading:

- Set outer loop variable `j` from 0 to `cols - 1` (iterating through columns).
- Set inner loop variable `i` from 0 to `rows - 1` (iterating through rows).
- Prompt user and read element into `arr[i][j]` using `scanf`.

7. Row-Major Order Printing using Single Loop:

- Set loop variable `k` from 0 to `(rows * cols) - 1`.
- Compute row index `r = k / cols`.
- Compute column index `c = k % cols`.
- Print element `arr[r][c]`.
- If `(k + 1) % cols == 0`, print a newline character to complete the row.

8. Stop

2. Source Code

```
#include <stdio.h>

int main() {
    int rows, cols;
    int i, j, k;

    // Reading rows with validation (must be positive)
    do {
        printf("Enter number of rows: ");
        scanf("%d", &rows);
        if (rows <= 0) {
            printf("Error: Number of rows must be positive. Try again.
");
        }
    } while (rows <= 0);

    // Reading columns with validation (must be positive)
```

```

do {
    printf("Enter number of columns: ");
    scanf("%d", &cols);
    if (cols <= 0) {
        printf("Error: Number of columns must be positive. Try again.
");
    }
} while (cols <= 0);

// Variable-length 2D array declaration
int arr[rows][cols];

// Reading elements column by column (Outer loop = columns, Inner loop = rows)
printf("
Enter array elements (column by column):
");
for (j = 0; j < cols; j++) {
    for (i = 0; i < rows; i++) {
        printf("Enter element for row %d, column %d: ", i + 1, j + 1);
        scanf("%d", &arr[i][j]);
    }
}

// Displaying elements in row-major order using a single loop and linear index
printf("
Array elements in Row-Major order:
");
for (k = 0; k < rows * cols; k++) {
    int r = k / cols;
    int c = k % cols;
    printf("%d\t", arr[r][c]);

    if ((k + 1) % cols == 0) {
        printf("
");
    }
}

return 0;
}

```

3. Test Table / Dry Run

Test Case: rows = 2, cols = 3. Inputs entered column by column: [0][0]=10, [1][0]=20, [0][1]=30, [1][1]=40, [0][2]=50, [1][2]=60.

k	r = k / cols	c = k % cols	Element `arr[r][c]`	Output	Condition `(k+1)%cols == 0`
0	0 / 3 = 0	0 % 3 = 0	arr[0][0]	10	1 % 3 != 0 (No newline)
1	0 / 3 = 0	1 % 3 = 1	arr[0][1]	30	2 % 3 != 0 (No newline)
2	0 / 3 = 0	2 % 3 = 2	arr[0][2]	50	3 % 3 == 0 (Print newline)
3	3 / 3 = 1	3 % 3 = 0	arr[1][0]	20	4 % 3 != 0 (No newline)
4	3 / 3 = 1	4 % 3 = 1	arr[1][1]	40	5 % 3 != 0 (No newline)
5	5 / 3 = 1	5 % 3 = 2	arr[1][2]	60	6 % 3 == 0 (Print newline)

4. Expected Output

```
Enter number of rows: 2
Enter number of columns: 3

Enter array elements (column by column):
Enter element for row 1, column 1: 10
Enter element for row 2, column 1: 20
Enter element for row 1, column 2: 30
Enter element for row 2, column 2: 40
Enter element for row 1, column 3: 50
Enter element for row 2, column 3: 60

Array elements in Row-Major order:
10 30 50
20 40 60
```

Question 2

1. Algorithm

1. **Start**
2. Declare integer variables `rows`, `cols`, `i`, `j`, and `index`.
3. Prompt user to enter values for `rows` and `cols`.
4. Declare 2D variable-length array `arr2D[rows][cols]`.
5. Declare 1D variable-length array `arr1D[rows * cols]`.
6. **Input 2D Array:**
 - Using nested loops (row outer, column inner), read elements into `arr2D[i][j]`.
7. **Copy 2D Array to 1D Array in Row-Major Order:**
 - Initialize `index = 0`.
 - Set outer loop variable `i` from 0 to `rows - 1`.
 - Set inner loop variable `j` from 0 to `cols - 1`.
 - Assign `arr1D[index] = arr2D[i][j]`.
 - Increment `index` by 1.
8. **Display 2D Array:**
 - Print 2D matrix using nested loops.
9. **Display 1D Array:**
 - Print 1D array sequentially using a single loop.
10. **Stop**

2. Source Code

```
#include <stdio.h>

int main() {
```

```

int rows, cols;
int i, j;

// Read dimensions
printf("Enter number of rows: ");
scanf("%d", &rows);
printf("Enter number of columns: ");
scanf("%d", &cols);

// Variable Length Array declarations
int arr2D[rows][cols];
int arr1D[rows * cols];

// Reading elements into 2D array
printf("
Enter elements for 2D array:
");
for (i = 0; i < rows; i++) {
    for (j = 0; j < cols; j++) {
        printf("Enter element arr2D[%d][%d]: ", i, j);
        scanf("%d", &arr2D[i][j]);
    }
}

// Copying 2D array to 1D array in row-major order
int index = 0;
for (i = 0; i < rows; i++) {
    for (j = 0; j < cols; j++) {
        arr1D[index] = arr2D[i][j];
        index++;
    }
}

// Displaying Original 2D Array
printf("
Original 2D Array:
");
for (i = 0; i < rows; i++) {
    for (j = 0; j < cols; j++) {
        printf("%d ", arr2D[i][j]);
    }
    printf("
");
}

// Displaying Copied 1D Array
printf("
Copied 1D Array (Row-Major Order):
");
for (i = 0; i < rows * cols; i++) {
    printf("%d ", arr1D[i]);
}
printf("
");

return 0;
}

```

3. Test Table / Dry Run

Test Case: rows = 2, cols = 2. 2D Array: `[[5, 10], [15, 20]]`

Row (i)	Col (j)	1D Index (`index`)	Assignment `arr1D[index] = arr2D[i][j]`	1D Array Status
0	0	0	<code>arr1D[0] = arr2D[0][0] = 5</code>	[5]
0	1	1	<code>arr1D[1] = arr2D[0][1] = 10</code>	[5, 10]
1	0	2	<code>arr1D[2] = arr2D[1][0] = 15</code>	[5, 10, 15]
1	1	3	<code>arr1D[3] = arr2D[1][1] = 20</code>	[5, 10, 15, 20]

4. Expected Output

```
Enter number of rows: 2
Enter number of columns: 2

Enter elements for 2D array:
Enter element arr2D[0][0]: 5
Enter element arr2D[0][1]: 10
Enter element arr2D[1][0]: 15
Enter element arr2D[1][1]: 20

Original 2D Array:
5 10
15 20

Copied 1D Array (Row-Major Order):
5 10 15 20
```